

Assignment 9

Author: Lan Stare Date: 21. 11. 2025

(a)

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

f1 = lambda x: np.sin(2 * np.pi * x) #example

def composite_trapezoidalquad(f, a, b, N):
    delta_x = (b - a) / N
    sum = 0
    for i in range(1, N + 1):
        if i == 1:
            sum += delta_x / 2 * (f(a) + f(delta_x * i))
        else:
            sum += delta_x / 2 * (f(delta_x * i) + f(delta_x * (i - 1)))
    return sum

print(composite_trapezoidalquad(f1, 0, 0.5, 100))

0.31828370581435783
```

(b)

```
In [2]: f2 = lambda x: x * np.sin(2 * np.pi * x)
a2 = 0
b2 = 1
exact = -1 / (2 * np.pi)
N2 = [2**i for i in range(1, 8)]

def error_and_plot(f, a, b, N_lst, exact_value):
    #first let's focus on the errors:
    errors = []
    h = [1 / n for n in N_lst]

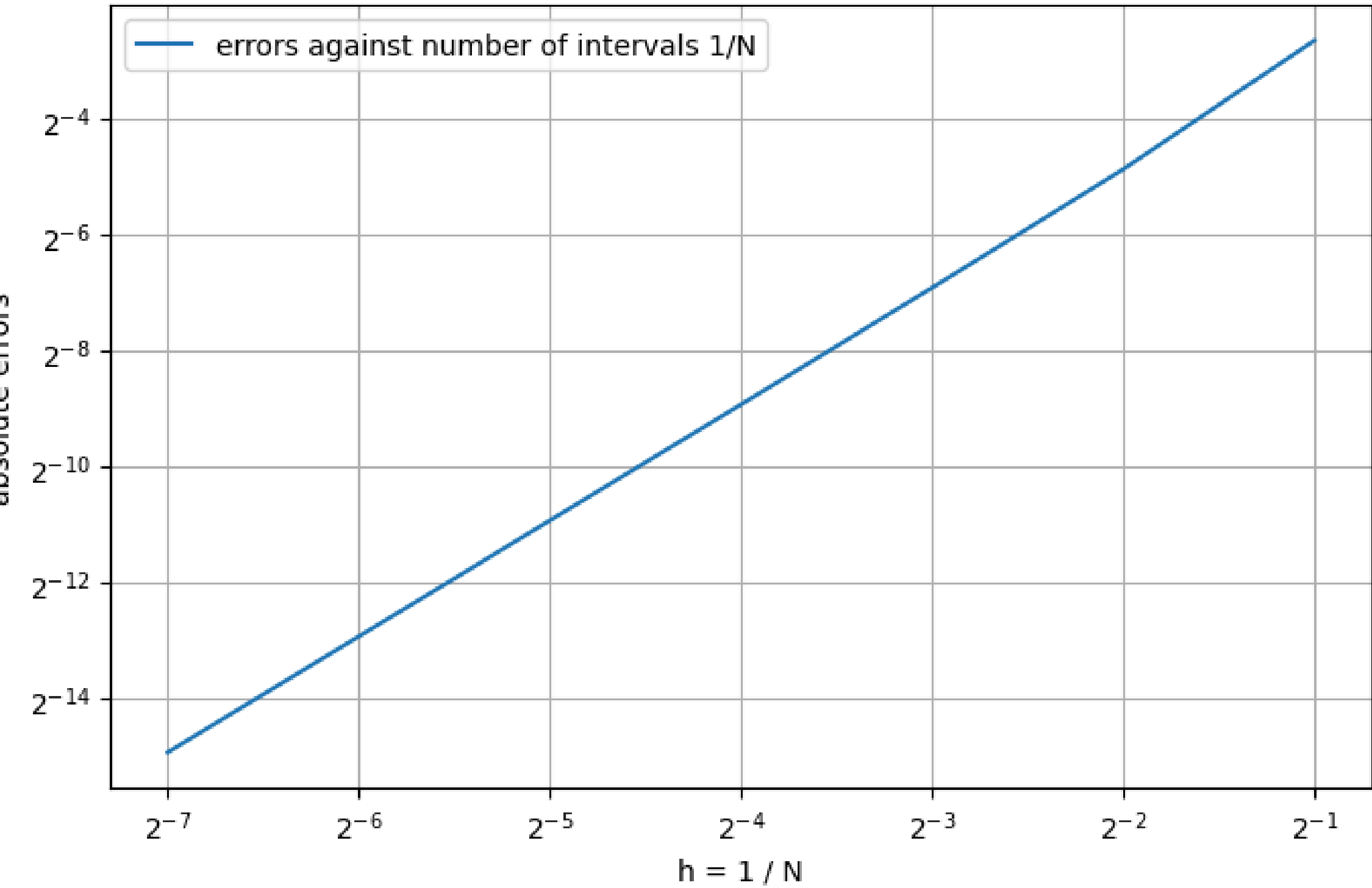
    #We'll need to plot the functions (errs vs delta_x) simultaneously:
    plt.figure(figsize=(8, 5))

    for element in N_lst:
        error = abs(composite_trapezoidalquad(f, a, b, element) - exact_value)
        errors.append(error)
        print(error)

    plt.loglog(h, errors, base=2, label="errors against number of intervals 1/N") #using base 2 because N has base 2 so we can better see the values expected values
    plt.xlabel("h = 1 / N")
    plt.ylabel("absolute errors")
    plt.legend()
    plt.grid()
    plt.show()

print(error_and_plot(f2, a2, b2, N2, exact))

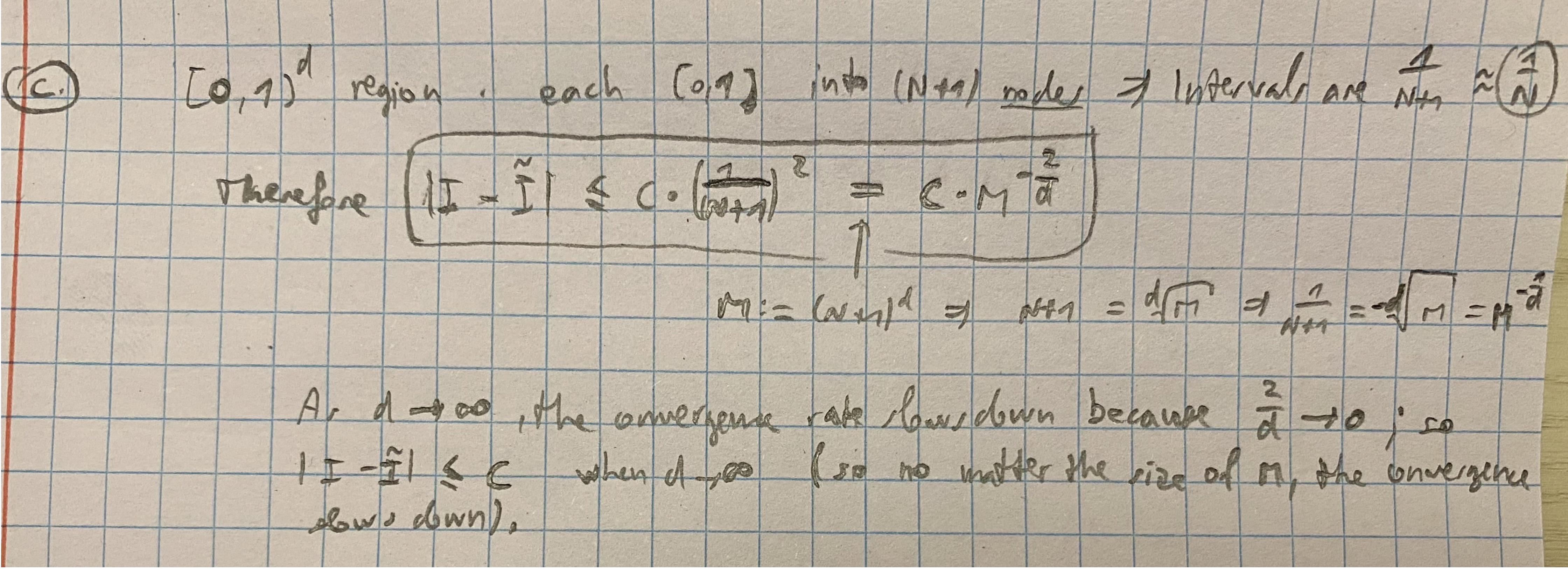
0.15915494309189532
0.034154943091895346
0.008266595443576913
0.0020505839629625655
0.0005116557855069093
0.00012785227168293778
3.195921656984346e-05
```



None

ANSWER: Our loglog plot is clearly linear, which is to be expected since we are approximating our function with a linear model (piecewise linear interpolation). If the number N doubles, the number of errors halves.

(c)



FEEDBACK (Correction):

h is the diagonal of the cube so $h^2 = d/N^2$, which makes the error go to infinity as d goes to infinity.

